Table of Contents

Claude Code runner v1.1 — Planscape template engine and workflow automation (S01–S18)

Companion to 20260423_planscape_template_engine_design_v1.1.docx. Paste this whole block to Claude Code after pulling the latest STING Tools repo. This runner replaces the v1.0 runner in full.

Operating rules (apply to every stage)

Flat file layout. All new .cs files sit at the repository root. No subfolder hierarchy. Namespaces: Planscape.Docs.Templates, Planscape.Docs.Workflow, Planscape.Docs.Search, Planscape.Docs.Signature.

Autocompact avoidance. Before any stage: git log --oneline -20 to see prior commits. Do not read BIMCoordinationCenter.cs, CLAUDE.md, ParamRegistry.cs, or any file over 50 KB in full. Use grep -n for locations and sed -n 'A,Bp' for ±30-line windows.

Editing pattern. str_replace for surgical edits. For new files, cat > filename.cs << 'EOF' heredoc split into 80–150 line chunks; append further chunks with cat >>.

Parameter discipline. Before adding any PRJ_* constant: grep -n "PRJ_" MR_PARAMETERS.txt ParamRegistry.cs. Reuse anything close enough.

Commit gate. End of every stage: dotnet build passes, then git add -A && git commit -m "<message>". No stage completes without a build-pass commit.

Resume protocol. git log --oneline -5 -- *.cs to find the last feat(docs): SNN or feat(workflow): SNN commit.

Scope discipline. Do not touch files outside each stage's stated targets.

Originator code. PLNS everywhere (not PSC). Company name default Planscape Limited.

Stage S01 — 13 PRJ_ORG_* parameters

Targets: ParamRegistry.cs, MR_PARAMETERS.txt.

Parameters to add (all Text unless noted, scoped to Project Information):

```
PRJ_ORG_PROJECT_CODE_TXT
PRJ_ORG_ORIGINATOR_CODE_TXT          (default "PLNS")
PRJ_ORG_COMPANY_NAME_TXT             (default "Planscape Limited")
PRJ_ORG_COMPANY_ADDRESS_TXT          (default "Kampala, Uganda")
PRJ_ORG_CLIENT_NAME_TXT
PRJ_ORG_APPOINTING_PARTY_TXT
PRJ_ORG_LEAD_APPOINTED_PARTY_TXT     (default "Planscape Limited")
PRJ_ORG_PARTICIPANTS_TXT
PRJ_ORG_PHASE_TXT                    (default "DE")
```

PRJ_ORG_CLASS_TXT (default "2")
PRJ_ORG_WORKFLOW_PROFILE_TXT (default "default")
PRJ_ORG_SIGNATURE_PROVIDER_TXT (default empty, meaning off)
PRJ_ORG_AI_EXTRACT_ENABLED_BOOL (Bool, default false)

Check for duplicates first:

```
grep -ni "PRJ_ORG_\|PROJECT_CODE\|ORIGINATOR\|APPOINTING_PARTY\|
LEAD_APPOINTED\|WORKFLOW_PROFILE" MR_PARAMETERS.txt ParamRegistry.cs
```

Generate UUIDv5 GUIDs. Choose a fixed namespace UUID for PRJ_ORG (a new one is fine; do not try to reuse PRJ_TB's namespace unless it is documented in the repo). Record the namespace in a comment block in ParamRegistry.cs:

```
python3 -c "
import uuid
ns = uuid.UUID('a7c0b2e4-4d91-4a55-9c7e-7f6e5d4c3b2a')   # Planscape docs namespace
for n in [

'PRJ_ORG_PROJECT_CODE_TXT','PRJ_ORG_ORIGINATOR_CODE_TXT','PRJ_ORG_C
OMPANY_NAME_TXT',

'PRJ_ORG_COMPANY_ADDRESS_TXT','PRJ_ORG_CLIENT_NAME_TXT','PRJ_ORG_AP
POINTING_PARTY_TXT',

'PRJ_ORG_LEAD_APPOINTED_PARTY_TXT','PRJ_ORG_PARTICIPANTS_TXT','PRJ_OR
G_PHASE_TXT',

'PRJ_ORG_CLASS_TXT','PRJ_ORG_WORKFLOW_PROFILE_TXT','PRJ_ORG_SIGNATU
RE_PROVIDER_TXT',
  'PRJ_ORG_AI_EXTRACT_ENABLED_BOOL'
]:
    print(n, str(uuid.uuid5(ns, n)))
"
```

Steps: 1. grep -n "TB_NOTES_LEGEND_REF_GUID\|AllTitleBlockParams" ParamRegistry.cs to locate insertion point. 2. str_replace to insert a new // ── Organisation parameters (v1.1 template engine + workflow) ── block with the 13 constants and their GUIDs immediately after the existing TB_ block. 3. Add public static readonly string[] AllOrganisationParams listing the 13 names. 4. Append 13 lines to MR_PARAMETERS.txt preserving UTF-8 BOM and CRLF. Grep an existing PRJ_TB_ line to confirm exact column layout. 5. dotnet build.

Commit: feat(docs): S01 — add 13 PRJ_ORG_* parameters for template engine and workflow

Verify:

```
grep -c "PRJ_ORG_" ParamRegistry.cs      # ≥ 26 (13 names + 13 GUIDs)
grep -c "PRJ_ORG_" MR_PARAMETERS.txt     # 13
dotnet build 2>&1 | grep -c error        # 0
```

Stage S02 — DeliverableRow extensions (v1.0 + v1.1 fields)

Target: BIMCoordinationCenter.cs.

Locate:

```
grep -n "internal class DeliverableRow\|public bool IsOverdue" BIMCoordinationCenter.cs
```

Add 12 v1.0 fields + 10 v1.1 fields + 3 new support classes:

v1.0 fields (from design doc):

```
public string DocNumber { get; set; }
public string Revision { get; set; }
public string FunctionalBreakdown { get; set; } = "ZZ";
public string SpatialBreakdown { get; set; } = "XX";
public string Originator { get; set; } = "PLNS";
public string RoleCode { get; set; }
public string ContractorRef { get; set; }
public string System { get; set; }
public string Subsystem { get; set; }
public string EquipmentType { get; set; }
public string IssuedBy { get; set; }
public string ReviewedBy { get; set; }
public string ApprovedBy { get; set; }
public string Supersedes { get; set; }
public string SupersededBy { get; set; }
public List<RevisionHistoryEntry> RevisionHistory { get; set; } = new();
public List<HoldEntry> Holds { get; set; } = new();
public List<ReferenceEntry> References { get; set; } = new();
```

v1.1 additions:

```
public string WorkflowState { get; set; }
public string AssignedTo { get; set; }
public DateTime? SlaDeadline { get; set; }
public List<WorkflowHistoryEntry> WorkflowHistory { get; set; } = new();
public string FileHashSha256 { get; set; }
public List<string> Tags { get; set; } = new();
public List<string> RelatedTransmittalIds { get; set; } = new();
public List<string> RelatedRfiIds { get; set; } = new();
public bool RequiresSignature { get; set; }
public string SignatureStatus { get; set; } = "None";
public string SignedFilePath { get; set; }
```

Support classes (insert immediately after DeliverableRow, before CoordLogEntry): RevisionHistoryEntry, HoldEntry, ReferenceEntry, WorkflowHistoryEntry.

Steps: 1. sed -n '505,525p' BIMCoordinationCenter.cs to view current closing. 2. str_replace the final property + closing brace with the full new set + brace. 3. Append support classes via str_replace anchoring on the CoordLogEntry class opening. 4. Find any JSON deserialisation of DeliverableRow and add null-safe defaults for new fields. 5. dotnet build.

Commit: feat(docs): S02 — extend DeliverableRow with ISO 19650 and workflow fields

Stage S03 — TemplateManifest POCOs

Target: new file TemplateManifest.cs. Namespace Planscape.Docs.Templates.

Structure: - TemplateManifest (root) - ProjectManifestBlock (project metadata, identifier format, schemes) - TemplateEntry (per-template definition) - ManifestExtensions (custom types/roles/suitability) - SignatureConfig (new in v1.1) - ValidationIssue (record)

Key methods: - static TemplateManifest Load(string path) - void Save(string path) - static TemplateManifest CreateDefault(Document doc) — populates from ProjectInformation + PRJ_ORG_* - List<ValidationIssue> Validate()

Default originator: "PLNS". **Default company name**: "Planscape Limited".

Split into 3 heredoc chunks of ~100 lines each. dotnet build.

Commit: feat(docs): S03 — add TemplateManifest POCOs

Stage S04 — DocumentIdentityGenerator

Target: new file DocumentIdentityGenerator.cs. Namespace Planscape.Docs.Templates.

Methods: - string Next(Document doc, TemplateManifest m, string type, string role, string fb, string sb) - string Preview(TemplateManifest m, string type, string role, string fb, string sb, int number) - int PeekNext(Document doc, TemplateManifest m, string type, string role, string fb, string sb) - List<int> Reserve(Document doc, TemplateManifest m, string type, string role, string fb, string sb, int count) (new in v1.1 — reserves a block for bulk operations)

Sequence store: _BIM_COORD/doc_sequences.json. Atomic write via .tmp + rename.

Format string parser: regex \{(\w+)(?::D(\d+))?\}. Support {number:D4}, {number:D6}.

Commit: feat(docs): S04 — add DocumentIdentityGenerator with reservation support

Stage S05 — TokenContext and TokenResolver

Targets: new files TokenContext.cs, TokenResolver.cs. Namespace Planscape.Docs.Templates.

TokenContext structure: - Dictionary<string, object> Doc - Dictionary<string, object> Project - Dictionary<string, object> People - Dictionary<string, object> Transmittal - Dictionary<string, List<Dictionary<string, object>>> Loops - Dictionary<string, object> AsDictionary() — flattens for MiniWord consumption (dotted keys)

Factories: - FromDeliverable(DeliverableRow d, Document doc, TemplateManifest m) - FromTransmittalRequest(TransmittalRequest r, ...)

TransmittalRequest: new DTO, kept in this file.

TokenResolver is thin with MiniWord: only used for pre-processing conditionals and validating tokens at manifest load. Methods: - List<string> FindAllTokens(string text) — regex \{\{([^}]+)\}\} - string Resolve(string token, TokenContext ctx) — used only by pre-processor - bool IsLoopStart(string s), bool IsLoopEnd(string s), string LoopName(string s)

Commit: feat(docs): S05 — add TokenContext and TokenResolver (MiniWord-compatible)

Stage S06 — MiniWordAdapter (replaces v1.0 hand-rolled renderer)

Targets: new file MiniWordAdapter.cs. Namespace Planscape.Docs.Templates.

Dependencies:

```
<PackageReference Include="MiniWord" Version="0.9.0" />
```

Add to StingTools.csproj: grep -n "PackageReference" StingTools.csproj to find the group and insert.

Class:

```
public static class MiniWordAdapter
{
    public static void Render(string templatePath, TokenContext ctx, string outputPath)
    {
        // 1. Pre-process: resolve {{#if}} conditionals by removing blocks with false conditions
        string processedTemplate = PreProcess(templatePath, ctx);

        // 2. Convert TokenContext to the Dictionary<string, object> MiniWord expects
        var dict = ctx.AsDictionary();

        // 3. Call MiniWord
        MiniSoftware.MiniWord.SaveAsByTemplate(outputPath, processedTemplate, dict);

        // 4. Post-process: resolve internal hyperlinks, adjust fields
        PostProcess(outputPath, ctx);

        // 5. Clean up the tmp template
        if (processedTemplate != templatePath && File.Exists(processedTemplate))
            File.Delete(processedTemplate);
    }

    private static string PreProcess(string templatePath, TokenContext ctx) { /* copy to tmp, resolve conditionals */ }
    private static void PostProcess(string outputPath, TokenContext ctx) { /* OpenXML post-work */ }
}
```

Pre-processor responsibilities: - Resolve {{#if doc.supersedes}}...{{/if}} conditional blocks - Flatten nested loops (MiniWord supports one level natively) - Ensure all {{token}} keys exist in the dictionary (use <TOKEN_NOT_FOUND:name> for missing) - Copy the template to a tmp file and modify that; original is untouched

Post-processor responsibilities: - Wire {{link:doc.supersedes}} to internal document references (add relationship + hyperlink) - Update PAGE and NUMPAGES fields if the document was modified - Set core properties (author = “Author” per style guide, last modified by = user)

Kept as safety net: LegacyDocxRenderer.cs — a minimal wrapper over OpenXml SDK for emergency fallback if MiniWord fails on a specific template. Used only when the manifest flag use_legacy_renderer: true is set.

Commit: feat(docs): S06 — MiniWordAdapter replacing hand-rolled renderer

Verify:

```
grep -n "MiniWord" StingTools.csproj        # package present
wc -l MiniWordAdapter.cs                    # ~150-250 lines
dotnet build 2>&1 | grep -c error           # 0
```

Stage S07 — XlsxTemplateRenderer (ClosedXML-based)

Target: new file XlsxTemplateRenderer.cs. Namespace Planscape.Docs.Templates.

Dependency:

```
<PackageReference Include="ClosedXML" Version="0.102.2" />
```

Class:

```
public static class XlsxTemplateRenderer
{
    public static void Render(string templatePath, TokenContext ctx, string outputPath)
    {
        using var workbook = new XLWorkbook(templatePath);
        foreach (var sheet in workbook.Worksheets)
        {
            ReplaceTokensInSheet(sheet, ctx);
            ExpandRowLoops(sheet, ctx);
        }
        workbook.SaveAs(outputPath);
    }
}
```

Loop mechanics: rows containing {{#loop_name}} mark the start; rows containing {{/loop_name}} mark the end. For each item, clone intermediate rows, substitute tokens, append below. Preserve cell styles, formats, and borders.

Formula adjustment: when rows are inserted, formulas referencing rows below the insertion point must shift. ClosedXML handles this automatically via InsertRowsBelow.

Commit: feat(docs): S07 — XlsxTemplateRenderer (ClosedXML)

Stage S08 — TemplateRegistry and TemplateEngine facade

Targets: new files TemplateRegistry.cs, TemplateEngine.cs. Namespace Planscape.Docs.Templates.

TemplateRegistry: - Load(string templatesDir, TemplateManifest m) - TemplateEntry ResolveById(string id) - TemplateEntry ResolveByPurpose(string family, string purpose) - List<ValidationIssue> ValidateAll()

TemplateEngine: - TemplateEngine(Document doc) - string RenderById(string id, TokenContext ctx) - string RenderByPurpose(string family, string purpose, TokenContext ctx) - Dispatch by extension: .docx → MiniWordAdapter, .xlsx → XlsxTemplateRenderer

Output path: _BIM_COORD/generated/YYYYMMDD_{doc_number}_{template_id}.{ext}.

Commit: feat(docs): S08 — TemplateRegistry and TemplateEngine facade

Stage S09 — DeliverableLifecycle

Target: new file DeliverableLifecycle.cs. Namespace Planscape.Docs.Templates.

Methods return LifecycleResult { DeliverableRow Updated; string TemplateId; string Message; }: - Issue(DeliverableRow d, Document doc, TemplateManifest m, string issuedBy, string reason) - ReIssue(DeliverableRow d, ...) - Publish(DeliverableRow d, ..., int stage) - Cancel(DeliverableRow d, ..., string reason) - Supersede(DeliverableRow d, string newDocNumber, ...) - Replace(DeliverableRow existing, DeliverableRow newReplacing, ...)

Each method: - Validates current state allows transition - Updates DocNumber, Revision, Suitability - Appends RevisionHistory row - Writes audit log entry (calls S16 AuditLog.Append) - Returns template ID for render

Persistence: _BIM_COORD/deliverables.json, preserves existing fields.

Commit: feat(docs): S09 — DeliverableLifecycle state machine

Stage S10 — TransmittalOrchestrator and path migration

Targets: new file TransmittalOrchestrator.cs; edits to BIMManagerCommands.cs, DocumentManagementDialog.cs.

TransmittalOrchestrator.Create(doc, request) returns TransmittalResult { JObject Record; string DocxPath; string TemplateId; string WorkflowInstanceId; }.

Pipeline: 1. Generate transmittal id 2. Build TokenContext.FromTransmittalRequest 3. Call TemplateEngine.RenderByPurpose("B", "transmittal", ctx) or ("C", "letter_transmittal", ctx) 4. Append to transmittals.json with new fields template_id, rendered_file_path, workflow_instance_id 5. Call WorkflowEngine.Start(workflowId, docId) (S15 — stub this for now, activate after S15) 6. Call AuditLog.Append("doc.rendered", ...) (S16 — stub) 7. Return result

Refactor CreateTransmittalCommand.Execute and DocumentManagementDialog.QuickTransmittal to delegate.

Commit: feat(docs): S10 — TransmittalOrchestrator with workflow and audit hooks

Stage S11 — Default template pack (six templates)

Deliverable: six .docx files in _template_sources/ folder, added as embedded resources.

Templates: - deliverable_standard.docx (A01) - deliverable_cancelled.docx (A02) - deliverable_superseded.docx (A03) - deliverable_replacing.docx (A04) - transmittal.docx (B06) - letter_transmittal.docx (C13)

Authoring: use Word directly. Replace all placeholders with {{token}}. For loops use MiniWord syntax: {{foreach items}} ... {{endforeach}}.

Planscape branding defaults (use these in every template): - Title: {{project.company_name}} (renders as “Planscape Limited” by default) - Logo placeholder: {{image:project.company_logo_path}} - Address line: {{project.company_address_line_1}} (“Kampala, Uganda” default) - Arial 11pt body, A4 portrait, 1440 DXA margins - Footer: PAGE of NUMPAGES centred, company name left, doc number right

Embed and extract: 1. Add to StingTools.csproj: xml <ItemGroup> <EmbeddedResource Include="_template_sources*.docx" /> </ItemGroup> 2. Create EmbeddedTemplates.cs: - ExtractIfMissing(Document doc) — on first run, extract each embedded template into _BIM_COORD/templates/ - ExtractDefaultManifest(Document doc) — write default manifest.json using TemplateManifest.CreateDefault - ExtractDefaultWorkflows(Document doc) — write default workflow JSONs (stub until S15) 3. Wire into BCC initialisation path so extraction runs once per project.

Commit: feat(docs): S11 — default template pack (six templates) with embedded extraction

Stage S12 — BCC Deliverables tab UI

Target: BIMCoordinationCenter.cs.

Append to BuildDeliverablesTab (line ~6736): - **Templates sub-section**: list registered templates, name, purpose, file, “Test render” button - **Identity sub-section**: display identifier_format, show DocumentIdentityGenerator.PeekNext for common type/role combinations - **Lifecycle actions**: per selected deliverable, show Issue/ReIssue/Publish/Cancel/Supersede/Replace buttons - **My queue sub-section** (v1.1 — activates once S15 ships): list workflow items assigned to current user with SLA countdown colour coding

Register actions: IssueDeliverable, ReIssueDeliverable, PublishDeliverable, CancelDeliverable, SupersedeDeliverable, ReplaceDeliverable.

Create DeliverableLifecycleCommands.cs with one Revit command class per action.

Update StingCommandHandler.cs command dispatch table.

Commit: feat(bcc): S12 — deliverable lifecycle UI in BCC Deliverables tab

Stage S13 — Document Manager integration

Target: DocumentManagementDialog.cs.

Steps: 1. grep -n "MakeDispatchBtn.*CreateTransmittal\|QuickTransmittal(" DocumentManagementDialog.cs 2. Add “Choose template” combo next to Create button (options B06 Transmittal, C13 Letter of transmittal) 3. Refactor QuickTransmittal to delegate to TransmittalOrchestrator passing the chosen template family 4. After create, show result dialog with “Open file” and “Reveal in folder” buttons using Process.Start 5. Add “Issue selected” button that calls DeliverableLifecycle.Issue in a loop for bulk operations (v1.1 bulk)

Commit: feat(docs): S13 — Document Manager uses TransmittalOrchestrator, bulk Issue

Stage S14 — Expanded template pack (ten more templates, three substages)

S14a — tabular deliverable + RFI + TQ

Author: - deliverable_tabular.xlsx (A05) — use ClosedXML authoring script if preferred - technical_query.docx (B07) - rfi.docx (B08)

Add manifest entries. Update EmbeddedTemplates include list.

Commit: feat(docs): S14a — tabular deliverable, RFI, TQ templates

S14b — MR, submittal, variation, technical response

Author: - material_requisition.docx (C10) — largest, uses MR-specific loops (scope_items, supplier_docs) - submittal_cover.docx (C11) - variation.docx (C12) - technical_response.docx (B09)

Commit: feat(docs): S14b — MR, submittal, variation, technical response templates

S14c — management templates

Author: - meeting_minutes.docx (D14) — triggered from BCC Meetings tab - progress_report.docx (D15) - handover_certificate.docx (D16)

Wire each to its trigger: Meetings tab “Export minutes” button, new “Generate progress report” command, handover added to Deliverables lifecycle.

Commit: feat(docs): S14c — meeting minutes, progress, handover templates

Stage S15 — Workflow engine

Targets: new files in namespace Planscape.Docs.Workflow: - WorkflowDefinition.cs — POCOs for states, transitions, escalations - WorkflowEngine.cs — runtime engine - WorkflowRegistry.cs — load from JSON files - SlaScanner.cs — opportunistic SLA checker

WorkflowDefinition structure:

```
public class WorkflowDefinition
{
    public string Id;
    public string Name;
    public string Description;
    public List<WorkflowState> States = new();
    public List<WorkflowTransition> Transitions = new();
    public string StartState;
}
public class WorkflowState
{
    public string Name;
    public int? SlaHours;
    public List<WorkflowEscalation> Escalations = new();
    public List<string> EntryActions = new();
    public List<string> ExitActions = new();
}
public class WorkflowTransition
{
    public string From;
    public string To;
    public string Action;          // e.g. "submit", "approve", "reject"
    public List<string> AllowedRoles = new();
    public string Condition;       // optional expression
```

```
}
public class WorkflowEscalation
{
    public int AfterHours;
    public string Action;          // remind / notify / reroute / auto_approve
    public string To;
}
```

WorkflowEngine methods: - Start(string workflowId, string docId) → creates WorkflowInstance in _BIM_COORD/workflow_state.json - Transition(string docId, string action, string byUser, string comment) → validates, applies, logs - GetInstance(string docId) → current state - GetMyQueue(string userEmail) → instances assigned to user, sorted by SLA urgency - CheckSlaBreaches() → scans all instances, returns list of breaches

SlaScanner: called on BCC open, tab switch, and every dispatch. Not a real-time timer. Fires EscalationActions when breaches detected.

Default workflow JSONs: ship five defaults in _workflow_sources/: - transmittal_default.json - rfi_default.json - tq_default.json - mr_default.json - deliverable_issue_default.json

Embed and extract like templates. EmbeddedTemplates.ExtractDefaultWorkflows(doc) now fully implemented.

Wire TransmittalOrchestrator.Create to call WorkflowEngine.Start. Add BCC “My queue” sub-section now that workflow data exists.

Commit: feat(workflow): S15 — workflow engine with SLA scanner and default workflows

Stage S16 — Audit log with tamper evidence

Target: new file AuditLog.cs in Planscape.Docs.Workflow.

Storage: _BIM_COORD/audit_log_{yyyy}_{MM}.jsonl — monthly rotation, append-only.

Entry schema:

```
{
  "ts": "2026-04-23T10:15:00.123Z",
  "user": "alice@planscape.co.ug",
  "action": "doc.issued",
  "doc_id": "TX-0042",
  "payload": { ... },
  "prev_hash": "sha256:...",
  "entry_hash": "sha256:..."
}
```

Each entry hashes {ts, user, action, doc_id, payload, prev_hash} to produce entry_hash. prev_hash of a new entry = entry_hash of the last entry in the file.

Methods: - AuditLog.Append(Document doc, string action, string docId, JObject payload) - List<AuditEntry> Read(Document doc, DateTime from, DateTime to, string actionFilter = null) - bool VerifyChain(Document doc, string fileOrMonth)

Retrofit: find every ProjectFolderEngine.LogActivity call and pair it with AuditLog.Append. Do not remove LogActivity — it remains for the existing Coord Log UI.

Coord Log UI update: existing Coord Log tab in BCC now reads from audit log (merged view with filters).

Commit: feat(workflow): S16 — audit log with tamper-evidence chain

Stage S17 — Document index and advanced search

Targets: new files in namespace Planscape.Docs.Search: - DocumentIndex.cs — Lucene.NET wrapper - SearchQueryBuilder.cs — faceted query construction

Dependency:

```
<PackageReference Include="Lucene.Net" Version="4.8.0-beta00016" />
<PackageReference Include="Lucene.Net.Analysis.Common" Version="4.8.0-beta00016" />
```

DocumentIndex methods: - Build(Document revitDoc) → reads document_register.json, builds in-memory index (FSDirectory on disk for large projects) - List<IndexedDoc> Search(string query, SearchFilters filters, int maxResults) - UpdateOne(IndexedDoc d) — on register write - Rebuild() — on register corruption

SearchFilters: type, role, status, direction, workflow_state, date range, tag list.

UI: extend Document Manager's filter bar with: - Free-text search box (top) - Faceted filter pills (type, role, status, workflow state) - "Saved searches" combo with user-saved queries in saved_searches.json - Sort by any column

Performance: target < 100 ms for 10 000 documents on typical dev workstation. Measure via Stopwatch and log warning if exceeded.

Commit: feat(search): S17 — Lucene-backed document index and advanced search UI

Stage S18 — Distribution groups and recipient matrix

Targets: new file DistributionGroups.cs in Planscape.Docs.Workflow; edits to DocumentManagementDialog.cs and BCC.

Storage: _BIM_COORD/distribution_groups.json (see design doc section 10.2).

DistributionGroups methods: - List<DistributionGroup> LoadAll(Document doc) - DistributionGroup SuggestFor(DeliverableRow d) — based on type/role/suitability - void Save(Document doc, List<DistributionGroup> groups)

UI: 1. In DocumentManagementDialog.QuickTransmittal, replace the freeform recipient text box with a picker showing suggested group(s) plus "Edit" to tweak members 2. Add "Recipient matrix" button at the BCC Deliverables tab — opens a new window showing documents × recipients grid with acknowledgement status 3. Add "Distribution groups" sub-tab under the existing Team Members tab for group management

Commit: feat(workflow): S18 — distribution groups and recipient matrix

Deferred to v1.2

S19: Signature provider abstraction + DocuSign/Adobe adapters (requires server-side key management)

S20: AI-assisted metadata extraction from incoming PDFs (requires server-side Python service)

Both tracked as design-complete, implementation deferred.

Final verification pass

```
# Template count
ls _template_sources/*.docx _template_sources/*.xlsx | wc -l      # 16

# Workflow count
ls _workflow_sources/*.json | wc -l                                # 5

# File count check (new .cs files)
ls *.cs | grep -E "^(TemplateManifest|TemplateRegistry|TokenContext|TokenResolver|
MiniWordAdapter|LegacyDocxRenderer|XlsxTemplateRenderer|TemplateEngine|
DocumentIdentityGenerator|DeliverableLifecycle|TransmittalOrchestrator|
EmbeddedTemplates|WorkflowDefinition|WorkflowEngine|WorkflowRegistry|SlaScanner|
AuditLog|DocumentIndex|SearchQueryBuilder|DistributionGroups|
DeliverableLifecycleCommands)\.cs$" | wc -l    # 21

# No hard-coded clients
grep -rniE "tilenga|totalenergies|sinopec" *.cs | grep -v "^[^:]*:\s*//" | head     # empty or only
comments
grep -rni "\"PSC\"" *.cs | head                                    # empty (should be PLNS)
grep -c "PLNS" ParamRegistry.cs EmbeddedTemplates.cs TemplateManifest.cs          # ≥ 3

# Full build
dotnet build 2>&1 | grep -E "error|warning:" | head

# MiniWord and ClosedXML present
grep -c "MiniWord\|ClosedXML\|Lucene" StingTools.csproj                  # ≥ 3
```

Final commit (if loose ends): chore(docs): S18 — final verification and cleanup

Summary of deliverables after S18

13 new PRJ_ORG_* shared parameters

Extended DeliverableRow with 22 new fields + 4 support classes

21 new .cs files at repo root

16 embedded template files (.docx + .xlsx)

5 embedded workflow JSON files

Default manifest, distribution groups, saved searches

BCC Deliverables tab with 4 new sub-sections (Templates, Identity, Lifecycle, My Queue)

Document Manager with advanced search, bulk issue, distribution groups

Audit log with tamper-evidence

MiniWord + ClosedXML + Lucene.NET dependencies

Zero hard-coded client branding

Originator code PLNS everywhere